Vibe Coding 进阶指南
从“野生对话”到“可控、可证、可复盘”的双驱动 AI 编程工作流

帮我优化一下

代码报错了怎么办

再加个新功能

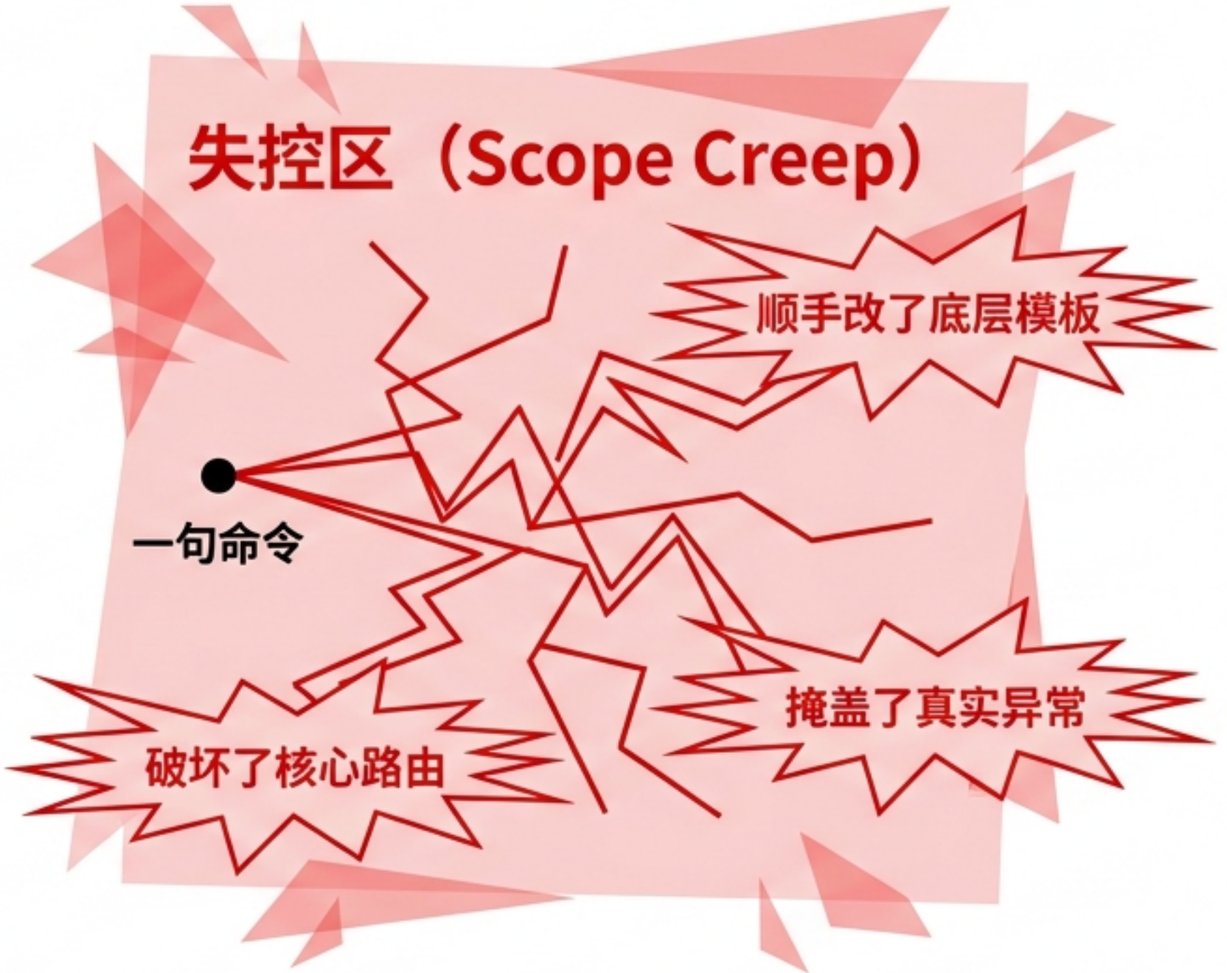


NotebookLM

AI 写代码越快，你越需要知道它到底改了什么

新手直觉：提示词 = 完整功能

一句命令 - - - - - - - - - - - - - - - - - - 完美代码



真实项目需要的是能落地的记录：为什么改，改哪些，不改哪些，怎样验证。聊天记录不能当项目历史

NotebookLM

破局之道：SDD + TDD 双驱动引擎



NotebookLM

双工具职责对照矩阵

维度	OpenSpec（规格侧）	Superpowers（执行侧）
定位	规格驱动 change 工作流	AI 的技能化开发方法
解决痛点	需求发散、边界失控、历史遗失	实现发散、缺测试、缺执行纪律
核心产物	proposal.md, specs/, design.md, tasks.md	Brainstorming 设计、TDD 计划、Review 结果
红色预警	✗ 不保证实现质量，不天然强制 TDD	✗ 不天然保存长期规格历史，不能替代需求事实源

NotebookLM

Vibe Coding 标准流水线



摒弃“一键生成”，建立可证、可复盘的流水线循环。

NotebookLM

Phase 1: 建立基线（看懂项目，而不是盲目重构）



顶层：业务模型
谁用？谁处理？

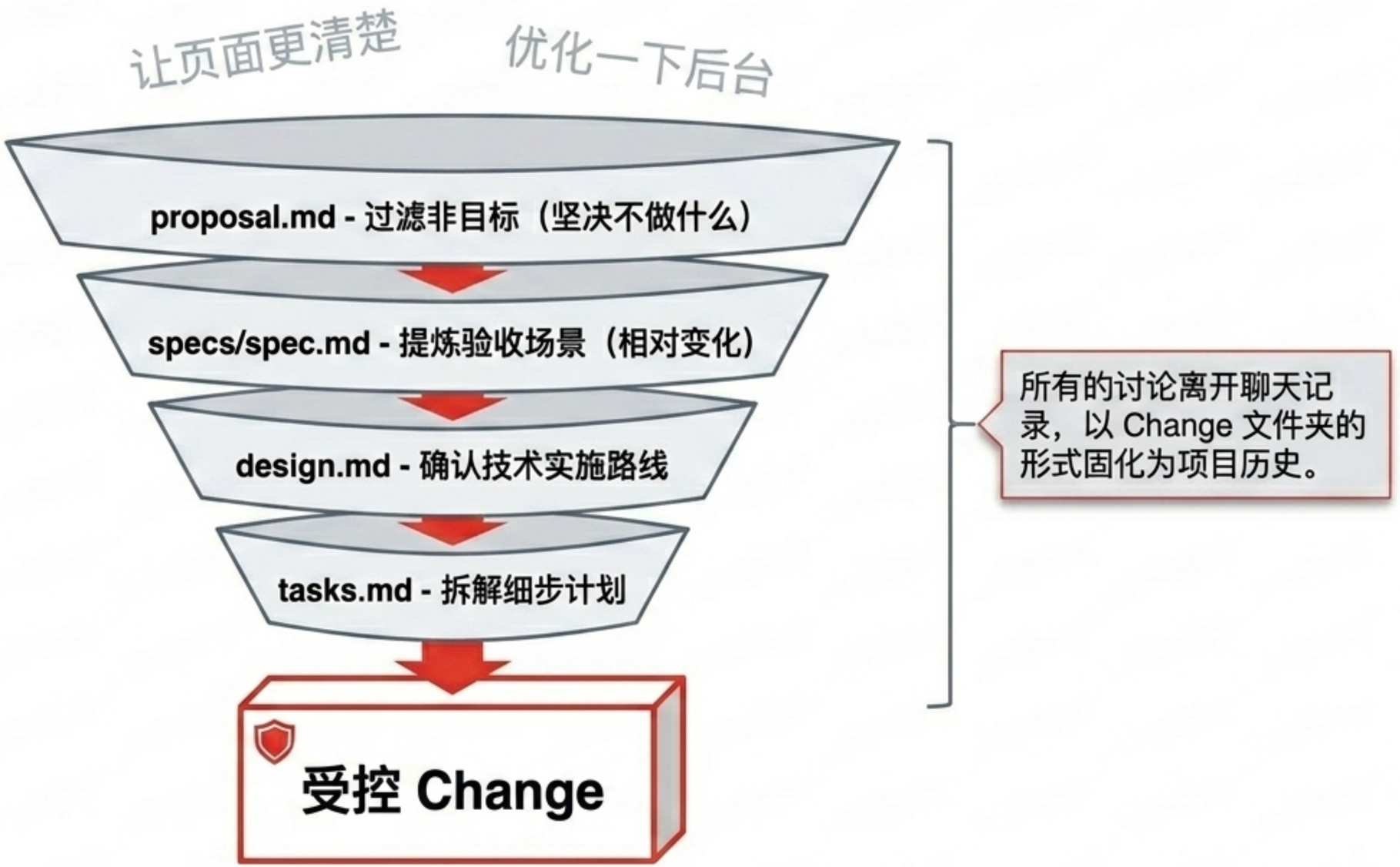
中层：目录结构
在哪？

底层：启动状态
能跑吗？记录启动命令与当前网页起点。

不动代码，先记起点

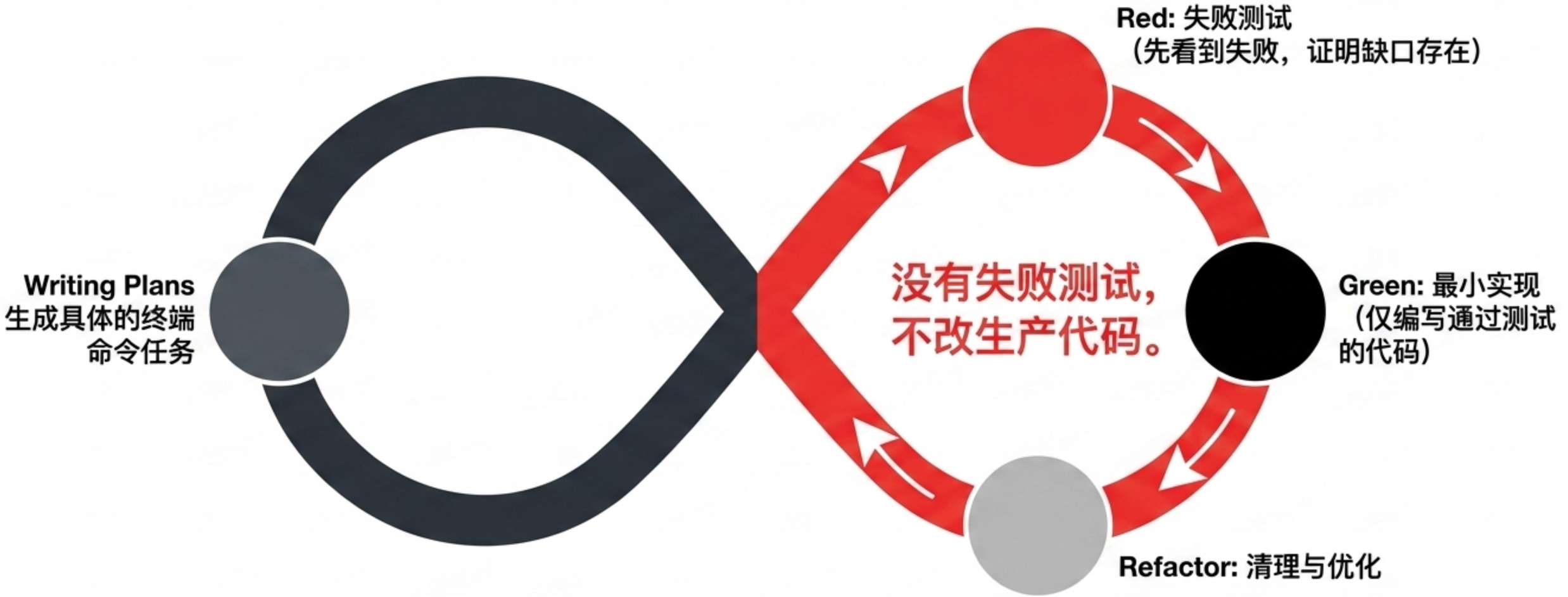
NotebookLM

Phase 2：用 OpenSpec 收束边界



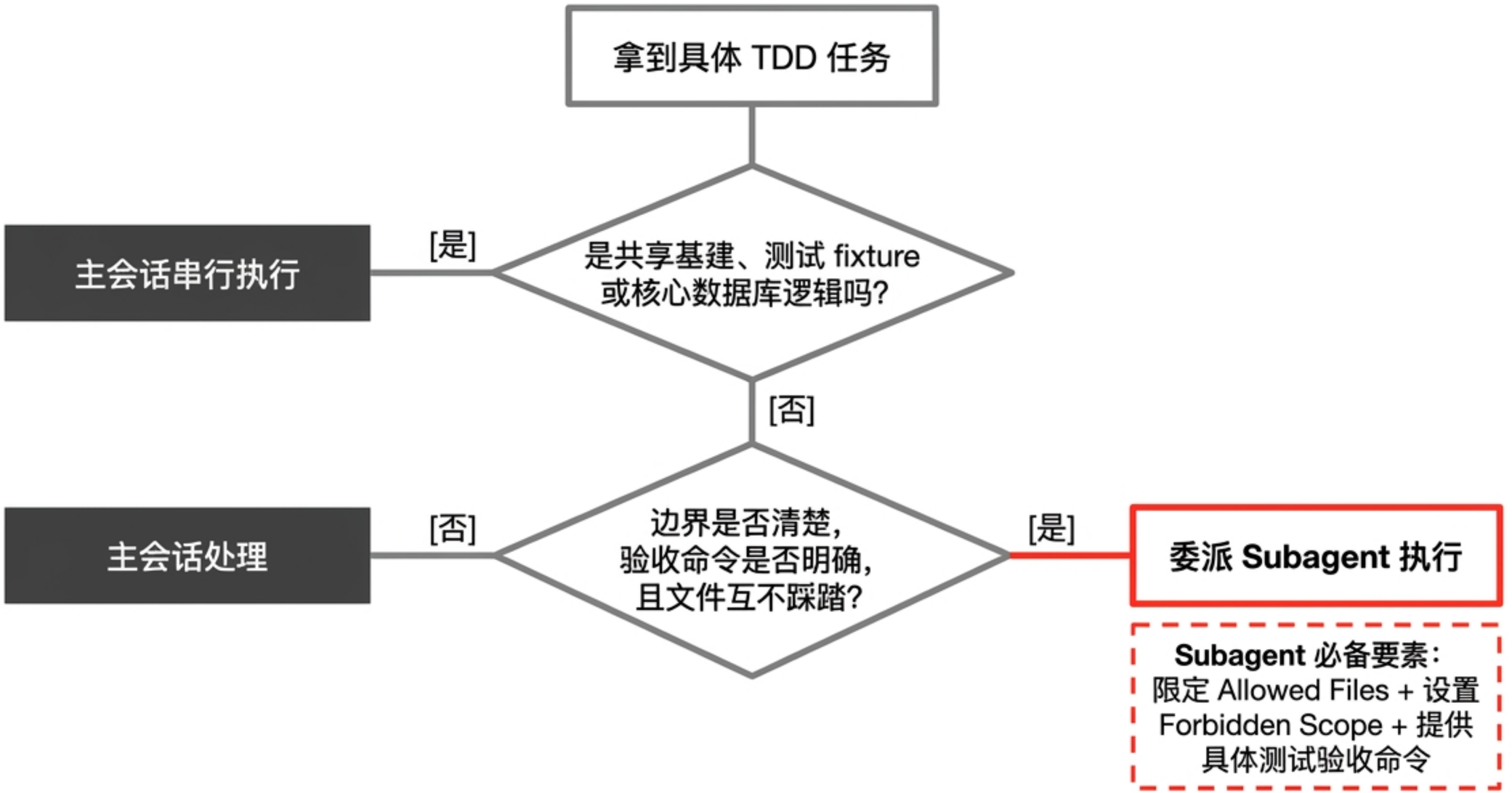
NotebookLM

Phase 3: Superpowers 执行纪律与 TDD 循环



NotebookLM

执行分流：Subagent 委派决策树



NotebookLM

审查的三重门

第一道门：Scope Review（防发散）

问题：改动还在 OpenSpec Change 和 Plan 范围内吗?
证据：真实 Plan 进度、AI 总结

第二道门：Diff Review（防违规）

问题：git diff 是否只包含了 Allowed Files? 没有掩盖异常?
证据：终端 git diff --stat 真实输出

第三道门：Completion Review（防假绿）

问题：聚焦测试全通过了吗? 网页 Evidence 拿到了吗?
证据：pytest 结果、网页截图比对



AI 不能自己宣布完成，
人类接管判断!

NotebookLM

Phase 5: 验证、校验与长期归档



Verify（功能完成证明）

测试通过、生成 Verification Report。证明代码确实满足了需求。



Validate（规格合规校验）

运行 openspec validate --strict。确保文档的格式、结构和验收标准全部合规。



Archive（合并不朽历史）

将已完成的 Change 收进长期 specs/，留下“当时为什么这么改”的永久追溯历史。

NotebookLM

真实项目优化的四级阶梯

阶梯 4：Config Hardening（配置硬化）
抽离硬编码配置。
系统健壮性升级。

阶梯 3：Visible UI（可见优化）
明确的视觉前后对比。
属于 Visible Change。

阶梯 2：Bugfix（真实缺陷修复）
纯后台稳定性提升。
属于 No Visible Change。

阶梯 1：Quality Baseline（质量基线）
铺设安全网。补齐测试。
属于 No Visible Change。

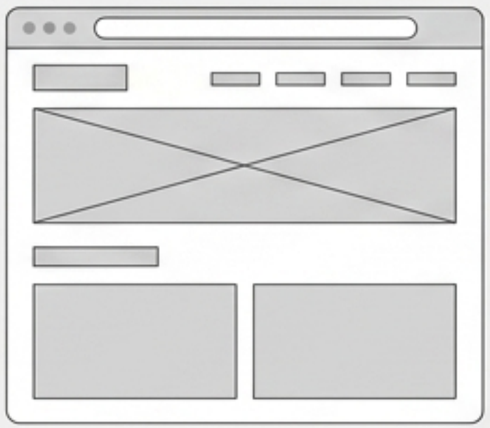
永远先建安全网，再修 Bug，最后改功能与 UI。不要在毫无保护的代码库里让 AI 裸奔。

NotebookLM

证据链：如何证明“看不见”的价值?

Visible Change: 页面优化

Before



After



证明方式：截图变化 + 指向具体的 OpenSpec 验收标准。

No Visible Change: 配置/基线/Bugfix



证明方式：测试 PASS 日志 + 无异常报告 + Verification Report。

无明显变化 ≠ 没做对

NotebookLM

神器剖析：Verification Report



执行溯源：运行的测试命令与具体结果

客观截图：网页证据（Before/After）本地路径链接

合规对照：OpenSpec 验收标准的逐条对照

工程现场：Blockers / 遗留风险记录

用这份结构化载体，代替零散、易丢失的 AI 聊天记录。

NotebookLM

Vibe Coding 进阶 Playbook

- 先看事实，再接受输出。永远检查 git diff，不要只听 AI 的文字总结。
- 锁定边界，拒绝发散。每次代码修改前，必须要有 OpenSpec 的明确范围。
- 遵循纪律，没有例外。没有看到失败测试（Red），就不允许 AI 修改生产代码。
- 大胆退回，人类裁决。当 AI 越界或测试假绿时，人类有绝对理由拒绝并重来。

Vibe Coding 依然利用 AI 的速度，但方向盘，永远在你手里。

NotebookLM